



Next-Generation Digital Processor (NDP) Interface Control Document

Version A

J. Dowell

February 2026

Approved:

Approved:

System Engineering

System Architecture

Date: _____

Date: _____

Long Wavelength Array Project

Change Record

Version	Date	Author(s)	Changes
A	2026-2-4	J. Dowell	Initial draft for the NDP system

Contents

Change Record	1
List of Tables	8
1 Description	9
1.1 Purpose	9
1.2 Scope	9
1.3 System Overview	9
1.3.1 Hardware Architecture	10
1.3.2 System Capacity	10
1.4 Observing Modes	11
1.4.1 DRX (Beam Recording)	11
1.4.2 COR (Correlator)	11
1.4.3 TBS (Transient Buffer – Streaming)	11
1.4.4 TBT (Transient Buffer – Triggered)	11
1.5 Document Organization	11
2 Abbreviations and Definitions	13
2.1 Abbreviations	13
2.2 Definitions	14
2.3 Fixed Parameters	14
3 Physical Interfaces	16
3.1 Overview	16
3.2 Signal Input from ASP	16
3.2.1 Input Specifications	16
3.3 ZCU102 F-engine Boards	16
3.3.1 Board Configuration	17
3.3.2 F-engine Processing	17
3.3.3 Network Interface	17
3.4 GPU Servers (X-engines)	17
3.4.1 Server Configuration	18
3.4.2 X-engine Processing	18
3.4.3 IPMI Interface	18
3.5 Headnode	18
3.6 Network Architecture	19
3.6.1 Data Network	19

3.6.2	Control Network	19
3.7	Output Data Interfaces	19
4	Monitor and Control Interface	20
4.1	Overview	20
4.2	Message Format	20
4.2.1	Command Message Format	20
4.2.2	Example Command Message	20
4.2.3	Response Message Format	21
4.3	Management Information Base (MIB)	21
4.3.1	General Information	21
4.3.2	System Configuration	22
4.3.3	DRX Configuration	22
4.3.4	TBS Configuration	22
4.3.5	Input Statistics	22
4.3.6	F-engine Board Status	22
4.3.7	X-engine Server Status	23
4.3.8	Global Status	23
4.3.9	Command Status	23
4.4	Control Commands	24
4.4.1	PNG — Ping	24
4.4.2	INI — Initialize	24
4.4.3	SHT — Shutdown	25
4.4.4	STP — Stop	26
4.4.5	DRX — Configure Beam Tuning	27
4.4.6	TBS — Transient Buffer – Streaming	28
4.4.7	TBT — Transient Buffer – Triggered	28
4.4.8	BAM — Beamformer Coefficients	29
4.4.9	COR — Correlator Configuration	30
4.4.10	RPT — Report MIB Entry	30
4.5	System States	30
5	Output Data Interfaces	32
5.1	Overview	32
5.2	Beamformer Output (DRX/DRX8)	32
5.2.1	Packet Header	32
5.2.2	Packet Payload	32
5.2.3	Data Rate	33
5.3	Correlator Output (COR)	33
5.3.1	Packet Header	33
5.3.2	Packet Payload	34
5.3.3	Data Rate	34
5.4	TBX Format Overview	35
5.4.1	TBX Packet Header	35
5.4.2	TBX Packet Payload	35
5.5	TBS Output (Transient Buffer – Streaming)	35
5.5.1	Configuration	35
5.5.2	Data Rate	35

5.6	TBT Output (Transient Buffer – Triggered)	36
5.6.1	Configuration	36
5.6.2	Local Storage Option	36
5.7	Output Summary	36
5.8	Internal Interfaces	37
5.8.1	ZCU102 F-engine Output	37
6	Safety Interface	39
6.1	Overview	39
6.2	Temperature Monitoring	39
6.2.1	Temperature Sensors	39
6.2.2	Temperature MIB Entries	39
6.3	IPMI Power Control	40
6.4	Subsystem State Machine	40
6.4.1	State Transitions	41
6.5	F-engine Board Monitoring	41
6.5.1	Input RMS Monitoring	41
6.6	X-engine Pipeline Monitoring	42
6.7	Timing Monitor Monitoring	42
6.8	Error Recovery	42
6.9	Command Validation	43
7	References	44
7.1	Reference Documents	44
7.2	Hardware References	44
7.3	Software Components	44
7.4	Pipeline Blocks	44
7.5	Configuration	48
7.5.1	Global Parameters	48
7.5.2	<code>mcs</code> — MCS Communication	48
7.5.3	<code>host</code> — System Hostnames	48
7.5.4	<code>ipmi</code> — IPMI Credentials	48
7.5.5	<code>fpga</code> — F-engine Configuration	48
7.5.6	<code>server</code> — X-engine Server Configuration	50
7.5.7	<code>drx</code> — X-engine Pipeline Configuration	50
7.5.8	<code>tengine</code> — T-engine Pipeline Configuration	50
7.5.9	<code>recorder</code> — Data Recorder Configuration	51
7.5.10	<code>tbt</code> — Transient Buffer Triggered Configuration	51
7.5.11	<code>log</code> — Logging Configuration	51
7.6	Source Code Repositories	51
A	Exit and Status Codes	53
A.1	Command Exit Codes	53
A.2	Subsystem Status Codes	53
B	Power Line Interference Flagger	54
B.1	Background	54
B.2	Scope of Flagging	54

B.3	Algorithm Overview	55
B.3.1	Stage 1: Mean Power Computation	55
B.3.2	Stage 2: Baseline Estimation and Burst Detection	55
B.3.3	Stage 3: Flagging and Replacement	55
B.4	Timing Considerations	56
B.5	Monitoring and Reporting	57
B.5.1	Statistics Tracking	57
B.5.2	Log Messages	57
B.5.3	System Health Integration	57
C	GPU Correlation with the Tensor Core Correlator	59
C.1	Overview	59
C.2	Tensor Core Correlator	59
C.3	Bifrost Wrapper	59
C.4	NDP Integration	61
C.4.1	Time-Axis Padding	61
C.4.2	Channel Decimation	61
C.4.3	Correlation and Accumulation	61
D	PFB Inversion in the T-Engine	63
D.1	Motivation	63
D.2	Approach	63
D.3	NDP Implementation	64
D.4	Effect on Output Data	64

List of Tables

1.1	F-engine Configuration	10
1.2	X-engine Configuration	10
1.3	T-engine Configuration	11
1.4	NDP System Capacity	11
2.2	NDP Fixed Parameters	15
3.1	Signal Input Specifications	16
3.2	ZCU102 Board Configuration	17
3.3	F-engine Signal Processing Parameters	17
3.4	F-engine Network Configuration	18
3.5	X-engine Server Configuration	18
3.6	IPMI Configuration	18
3.7	Headnode Configuration	19
3.8	Output Data Interfaces	19
4.1	Command Message Format	20
4.2	Response Message Format	21
4.3	General MIB Entries	21
4.4	System Configuration MIB Entries	22
4.5	DRX Configuration MIB Entries	22
4.6	TBS Configuration MIB Entries	22
4.7	Input Statistics MIB Entries	23
4.8	F-engine Board MIB Entries	23
4.9	X-engine Server MIB Entries	23
4.10	Global Status MIB Entries	24
4.11	Command Status MIB Entry	24
4.12	PNG Command	24
4.13	INI Command	25
4.14	INI Command Options	25
4.15	SHT Command	26
4.16	SHT Command Options	26
4.17	STP Command	26
4.18	STP Mode Strings	27
4.19	DRX Command	27
4.20	DRX Command Data Format (Big-Endian)	27
4.21	DRX Filter Codes	28

4.22	TBS Command	28
4.23	TBS Command Data Format (Big-Endian)	28
4.24	TBS Filter Codes	29
4.25	TBT Command	29
4.26	TBT Command Data Format (Big-Endian)	29
4.27	BAM Command	30
4.28	BAM Command Data Format (Big-Endian)	30
4.29	COR Command	30
4.30	COR Command Data Format (Big-Endian)	31
4.31	RPT Command	31
4.32	System Status Values	31
5.1	DRX/DRX8 Packet Header (32 bytes)	33
5.2	DRX Frame Count Word Bit Fields	33
5.3	COR Packet Header	34
5.4	COR Frame Count Word Bit Fields	34
5.5	TBX Packet Header (TBS/TBT) — 28 bytes	35
5.6	TBS Configuration	36
5.7	TBT Configuration	36
5.8	NDP Output Summary	36
5.9	ZCU102 Packet Header (32 bytes)	37
5.10	ZCU102 Payload Format	37
6.1	Temperature Monitoring MIB Entries	40
6.2	IPMI Power Operations	40
6.3	NDP State Machine	40
7.1	NDP Software Modules	45
7.2	X-engine Pipeline Blocks (<code>ndp_drx.py</code>)	45
7.3	T-engine Pipeline Blocks (<code>ndp_tengine.py</code>)	45
7.4	Top-level Configuration Parameters	48
7.5	<code>mcs</code> Configuration Parameters	48
7.6	<code>host</code> Configuration Parameters	49
7.7	<code>ipmi</code> Configuration Parameters	49
7.8	<code>fpga</code> Configuration Parameters	49
7.9	<code>server</code> Configuration Parameters	50
7.10	<code>drx[i]</code> Configuration Parameters	50
7.11	<code>tengine[i]</code> Configuration Parameters	51
7.12	<code>recorder[i]</code> Configuration Parameters	51
7.13	<code>tbt</code> Configuration Parameters	51
7.14	<code>log</code> Configuration Parameters	52
A.1	Command Exit Codes	53
B.1	Flagger Time Window Parameters	55
B.2	Quantization Correction Polynomial Coefficients	56
B.3	Power Line Flagger Timing	57
C.1	TCC Time-Axis Constraints by Input Bit Depth	60

C.2 Bifrost TCC Wrapper Array Shapes	60
C.3 Correlator Channel Budget (Default Configuration)	61

Section 1

Description

1.1 Purpose

This document describes the interface control for the Next-Generation Digital Processor (NDP) subsystem of the Long Wavelength Array at the New Mexico site (LWA-NA). The NDP is responsible for digitizing, channelizing, and processing the analog signals from the Front End Electronics (FEE) via the Analog Signal Processor (ASP), producing beamformed and correlated data products.

1.2 Scope

This Interface Control Document (ICD) defines:

- Physical interfaces to and from NDP
- Monitor and control (M&C) interfaces
- Output data interfaces and formats
- Safety interfaces
- Command and response message formats
- Management Information Base (MIB) entries

1.3 System Overview

The NDP subsystem performs the following functions:

- Analog-to-digital conversion at 196 MHz sample rate
- F-engine processing: Polyphase filter bank (PFB) channelization with 4096 channels
- X-engine processing: Power line interference flagging (Appendix B)
- X-engine processing: Beamforming (up to 4 dual-polarization beams)
- X-engine processing: Correlation (full visibility matrix)

- X-engine processing: Streaming complex spectra output from all antennas (TBS)
- X-engine processing: Triggered complex spectra dumps from all antennas (TBT)
- T-engine processing: Conversion from frequency domain to time domain for DRX output

1.3.1 Hardware Architecture

The NDP system consists of three main hardware components:

ZCU102 F-engine Boards

The F-engines perform analog-to-digital conversion and frequency channelization using Xilinx ZCU102 FPGA boards. Table 1.1 summarizes the F-engine configuration.

Table 1.1: F-engine Configuration

Parameter	Value
Number of boards	16
Inputs per board	32 (16 stands $\times$ 2 polarizations)
Sample rate	196 MHz
ADC resolution	8 bits
FFT channels	4096
Channel bandwidth	23.926 kHz

GPU Servers (X-engines)

The X-engines perform beamforming and correlation using GPU-accelerated servers running the Bifrost framework. Table 1.2 lists the X-engine parameters.

Table 1.2: X-engine Configuration

Parameter	Value
Number of servers	4
Pipelines per server	4
Total pipelines	16
Processing framework	Bifrost

T-engines

The T-engines convert frequency-domain beamformed data back to the time domain for DRX-format output, producing Mark 5C compatible packets. Table 1.3 lists the T-engine parameters.

1.3.2 System Capacity

Table 1.4 summarizes the overall system capacity.

Table 1.3: T-engine Configuration

Parameter	Value
Number of servers	1
Pipelines per server	4 (one per beam)
Total pipelines	4
Processing framework	Bifrost

Table 1.4: NDP System Capacity

Parameter	Value
Maximum stands	256
Polarizations per stand	2
Total inputs	512
Maximum beams	4 (dual-polarization)
Frequency channels	4096
Maximum tunings per beam	2

1.4 Observing Modes

1.4.1 DRX (Beam Recording)

The primary observing mode provides up to 4 dual-polarization beams with configurable tuning frequencies. Each beam can have up to 2 independent frequency tunings. The DRX output is in Mark 5C compatible format.

1.4.2 COR (Correlator)

The correlator mode cross-correlates all 512 inputs to produce a full visibility matrix using the Tensor Core Correlator (Appendix C). The integration time is configurable via the `COR` command.

1.4.3 TBS (Transient Buffer – Streaming)

Continuous streaming mode that outputs frequency-domain spectra for all 512 inputs. TBS is designed for 100% duty cycle operation with a narrow bandwidth selection (~ 200 kHz at filter code 8). This mode can run concurrently with other observing modes.

1.4.4 TBT (Transient Buffer – Triggered)

Triggered dump mode that captures frequency-domain spectra with wider bandwidth than TBS. TBT trades duty cycle for increased bandwidth, making it suitable for capturing transient events with more spectral coverage.

1.5 Document Organization

This document is organized as follows:

- **Section 1** (this section): General description and scope

- **Section 2:** Abbreviations and definitions
- **Section 3:** Physical interface specifications
- **Section 4:** Monitor and control interface
- **Section 5:** Output data interfaces
- **Section 6:** Safety interface
- **Section 7:** References

Section 2

Abbreviations and Definitions

2.1 Abbreviations

Abbreviation	Definition
ADC	Analog-to-Digital Converter
ASP	Analog Signal Processor
BAM	Beamformer coefficient command
COR	Correlator command/mode
DRX	Digital Receiver command/mode
FEE	Front End Electronics
FFT	Fast Fourier Transform
FPGA	Field-Programmable Gate Array
GPU	Graphics Processing Unit
ICD	Interface Control Document
IPMI	Intelligent Platform Management Interface
LWA	Long Wavelength Array
M&C	Monitor and Control
MCS	Monitor and Control System
MIB	Management Information Base
MJD	Modified Julian Date
MPM	Milliseconds Past Midnight
NDP	Next-Generation Digital Processor
PFB	Polyphase Filter Bank
TBS	Transient Buffer – Streaming
TBT	Transient Buffer – Triggered
TCC	Tensor Core Correlator
UDP	User Datagram Protocol
ZCU	Zynq UltraScale+ MPSoC (ZCU102 board)

2.2 Definitions

Stand A single antenna structure consisting of two polarizations (X and Y). Each stand is identified by a unique number from 1 to 256.

Input A single signal path from one polarization of one stand. Inputs are numbered 1 to 512, where input n corresponds to stand $\lceil n/2 \rceil$ and polarization $(n - 1) \bmod 2 + 1$.

Board A ZCU102 FPGA board that services 16 stands (32 inputs). Boards are numbered 1 to 16.

Server A GPU-equipped processing server running the X-engine software. Servers are numbered 1 to 4.

Head node A GPU-equipped processing server running the NDP control and T-engine software.

Pipeline A processing instance within a server. Each server runs 4 pipelines.

Beam A phased-array beam output. Beams are numbered 1 to 4, each with two polarizations.

Tuning A frequency setting for a beam. Each beam supports up to 2 tunings.

Channel A frequency channel produced by the PFB. Channels are numbered 0 to 4095, with channel n centered at frequency $n \times f_c$ Hz, where $f_c \approx 23.926$ kHz (196000 kHz / 8192).

Subslot A time subdivision of a second. There are 100 subslots per second (10 ms each).

Slot A one-second time interval used for command scheduling.

2.3 Fixed Parameters

Site-Specific Configuration

The parameters shown below are for a full station like LWA1. For a mini-station the values for NSTAND and NINPUT will be one-fourth of the presented values. There may also be other differences in NPIPE_PER_SERVER.

Table 2.2 lists the fixed parameters used throughout this document.

Table 2.2: NDP Fixed Parameters

Symbol	Value	Description
SUBSYSTEM	“NDP”	Subsystem identifier
f_s	196.0 MHz	Signal sampling frequency
f_c	23.926 kHz	Channel bandwidth ($f_s/(2 \times 4096)$)
NCHAN	4096	Number of frequency channels
NSTAND	256	Number of stands
NPOL	2	Polarizations per stand
NINPUT	512	Total number of inputs
NBEAM	4	Maximum number of beams
NBOARD	16	Number of F-engine boards
NSERVER	4	Number of X-engine servers
NPIPE_PER_SERVER	4	Pipelines per server
ADC_BITS	8	ADC resolution

Section 3

Physical Interfaces

3.1 Overview

The NDP subsystem interfaces with the following components:

- Analog Signal Processor (ASP) — signal input
- Monitor and Control System (MCS) — control interface
- Data recorders — output data
- Network infrastructure — data transport

3.2 Signal Input from ASP

The NDP receives conditioned analog signals from the ASP via coaxial cables. Each stand provides two polarization signals (X and Y).

3.2.1 Input Specifications

Table 3.1 lists the signal input specifications.

Table 3.1: Signal Input Specifications		
Parameter	Specification	Notes
Total inputs	512	256 stands $\times$ 2 polarizations
Connector Type	QMA Female	
Impedance	50 Ω	

3.3 ZCU102 F-engine Boards

The F-engines perform analog-to-digital conversion and frequency channelization using Xilinx ZCU102 FPGA boards that each host two LWA digitizer boards.

3.3.1 Board Configuration

Table 3.2 summarizes the ZCU102 board configuration.

Table 3.2: ZCU102 Board Configuration

Parameter	Value
Number of boards	16
Inputs per board	32
Stands per board	16
Sample rate	196 MHz
ADC resolution	8 bits

Site-Specific Configuration

Currently the LWA-NA site uses SNAP2 boards instead of the ZCU102. The SNAP2s can host four LWA digitizer boards each. The other parameters are the same as with the ZCU102.

3.3.2 F-engine Processing

Each ZCU102 board performs the following processing:

1. Analog-to-digital conversion at 196 MHz with 8-bit resolution
2. Polyphase filter bank (PFB) with 4096 channels
3. Equalization (gain adjustment per channel)
4. Packetization and transmission to X-engines

Table 3.3 gives the signal processing parameters for each F-engine board.

Table 3.3: F-engine Signal Processing Parameters

Parameter	Value
FFT size	8192 (4096 output channels)
Channel bandwidth	23.926 kHz
Total bandwidth	98 MHz
Output data format	Complex 8-bit (4+4)

3.3.3 Network Interface

Each ZCU102 board has a 40 Gbps Ethernet interface for transmitting channelized data to the X-engine servers. Table 3.4 lists the network configuration.

3.4 GPU Servers (X-engines)

The X-engines perform beamforming and correlation using GPU-accelerated servers.

Table 3.4: F-engine Network Configuration

Parameter	Value
Interface speed	40 Gbps
Protocol	UDP
Hostnames	zcu01–zcu16

3.4.1 Server Configuration

Table 3.5 summarizes the X-engine server configuration.

Table 3.5: X-engine Server Configuration

Parameter	Value
Number of servers	4
Pipelines per server	4
Hostnames	ndp1–ndp4
Processing framework	Bifrost

3.4.2 X-engine Processing

Each X-engine pipeline performs:

- Packet capture and reordering
- Beamforming (delay and sum with gain control)
- Correlation (cross-multiplication and accumulation)
- Triggered buffer capture
- Output packetization

3.4.3 IPMI Interface

Each server includes an IPMI interface for out-of-band management. Table 3.6 lists the IPMI settings.

Table 3.6: IPMI Configuration

Parameter	Value
IPMI hostnames	ndp1-ipmi – ndp4-ipmi
Capabilities	Power control, sensor monitoring

3.5 Headnode

The headnode (hostname: **ndp**) runs the NDP control service and interfaces between the system and MCS. Table 3.7 lists the headnode parameters.

Table 3.7: Headnode Configuration

Parameter	Value
Hostname	ndp
Control service	ndp-control
MCS receive port	1742
MCS send port	1743

3.6 Network Architecture

3.6.1 Data Network

The data network carries channelized data from F-engines to X-engines:

- 40 Gbps (F-engines) and 100 Gbps (X-engines) Ethernet connections
- Dedicated data switch
- Multicast or unicast UDP packets

3.6.2 Control Network

The control network provides:

- SSH access to all systems
- IPMI management access
- MCS command interface

3.7 Output Data Interfaces

Output data from NDP is transmitted via UDP packets. Table 3.8 summarizes the output data interfaces.

Table 3.8: Output Data Interfaces

Output Type	Format	Protocol	Destination
DRX/PBEAM	Mark 5C	UDP	Data recorder
COR	Custom	UDP	Orville Imager
TBS	Custom	UDP	Data recorder
TBT	Custom	UDP/File	Data recorder <i>or</i> Local storage

Section 4

Monitor and Control Interface

4.1 Overview

The NDP subsystem communicates with the Monitor and Control System (MCS) via UDP messages. Commands are received on port 1742 and responses are sent to port 1743.

4.2 Message Format

4.2.1 Command Message Format

Commands from MCS to NDP follow the standard LWA message format (Table 4.1).

Table 4.1: Command Message Format

Field	Bytes	Position	Description
Destination	3	0–2	Subsystem ID (“NDP”)
Sender	3	3–5	Sender ID (“MCS”)
Command	3	6–8	Command code
Reference	9	9–17	Reference number
Data Length	4	18–21	Length of data section
MJD	6	22–27	Modified Julian Date
MPM	9	28–36	Milliseconds Past Midnight
-	1	37	Space
Data	Variable	38+	Command-specific data

4.2.2 Example Command Message

A DRX command from MCS to NDP:

NDP MCS DRX 123 15 60710 43200000 <binary data>

Where:

- Destination = “NDP” (padded to 3 chars)
- Sender = “MCS” (padded to 3 chars)

- Command = “DRX” (padded to 3 chars)
- Reference = 123 (right-justified in 9 chars)
- Data Length = 15 (right-justified in 4 chars)
- MJD = 60710 (right-justified in 6 chars)
- MPM = 43200000 (right-justified in 9 chars)
- Followed by 15 bytes of binary command data

4.2.3 Response Message Format

Responses from NDP to MCS include the command status and any requested data (Table 4.2).

Table 4.2: Response Message Format

Field	Bytes	Position	Description
Destination	3	0–2	Destination ID (echo of sender)
Sender	3	3–5	“NDP”
Command	3	6–8	Echo of command
Reference	9	9–17	Echo of reference number
Data Length	4	18–21	Length of data section
MJD	6	22–27	Response MJD
MPM	9	28–36	Response MPM
-	1	37	Space
Accept/Reject	1	38	“A” (accept) or “R” (reject)
Status	7	39–45	Status code
Data	Variable	46+	Response data

4.3 Management Information Base (MIB)

The following MIB entries can be queried using the RPT command.

4.3.1 General Information

Table 4.3 lists the general MIB entries that identify the subsystem and report its overall status.

Table 4.3: General MIB Entries

MIB Entry	Type	Description
SUMMARY	char[7]	Current system status
INFO	char[256]	Detailed status information
LASTLOG	char[256]	Last log entry
SUBSYSTEM	char[3]	Subsystem identifier (“NDP”)
SERIALNO	char[5]	Serial number
VERSION	char[256]	Software version

4.3.2 System Configuration

Table 4.4 lists the fixed system configuration parameters.

Table 4.4: System Configuration MIB Entries

MIB Entry	Type	Description
NUM_STANDS	uint16	Number of stands (256)
NUM_SERVERS	uint8	Number of servers (4)
NUM_BOARDS	uint8	Number of F-engine boards (16)
NUM_BEAMS	uint8	Number of beams (4)
NUM_DRX_TUNINGS	uint8	Tunings per beam (2)
NUM_FREQ_CHANS	uint16	Number of frequency channels (4096)
STAT_SAMPLE_SIZE	uint32	Samples used for statistics (512)

4.3.3 DRX Configuration

Table 4.5 lists the per-tuning DRX configuration entries, where $\{n\}$ ranges over active tunings.

Table 4.5: DRX Configuration MIB Entries

MIB Entry	Type	Description
DRX $\{n\}$ _CONFIG_FREQ	float64	Tuning $\{n\}$ frequency in Hz
DRX $\{n\}$ _CONFIG_FILTER	uint16	Tuning $\{n\}$ filter code
DRX $\{n\}$ _CONFIG_GAIN	uint16	Tuning $\{n\}$ gain setting
DRX $\{n\}$ _CONFIG_HIGH_DR	uint8	Tuning $\{n\}$ high DR mode

4.3.4 TBS Configuration

Table 4.6 lists the TBS configuration entries.

Table 4.6: TBS Configuration MIB Entries

MIB Entry	Type	Description
TBS_CONFIG_FREQ	float64	TBS center frequency in Hz
TBS_CONFIG_FILTER	uint16	TBS filter code

4.3.5 Input Statistics

Table 4.7 lists the per-input statistics entries, where $\{n\}$ is the 1-based input index (stand $\times$ polarization).

4.3.6 F-engine Board Status

Table 4.8 lists the per-board F-engine status entries, where $\{n\}$ is the 1-based board index.

Table 4.7: Input Statistics MIB Entries

MIB Entry	Type	Description
ANT{n}_RMS	float32	RMS of samples for input {n}
ANT{n}_DCOFFSET	float32	Mean (DC offset) of samples
ANT{n}_SAT	int32	Saturated sample count
ANT{n}_PEAK	int32	Peak sample value

Table 4.8: F-engine Board MIB Entries

MIB Entry	Type	Description
BOARD{n}_STAT	char[256]	Board {n} status (JSON)
BOARD{n}_INFO	char[256]	Board {n} information (JSON)
BOARD{n}_TEMP_MAX	float32	Maximum temperature
BOARD{n}_TEMP_MIN	float32	Minimum temperature
BOARD{n}_TEMP_AVG	float32	Average temperature
BOARD{n}_FIRMWARE	char[256]	Firmware version
BOARD{n}_HOSTNAME	char[256]	Network hostname

4.3.7 X-engine Server Status

Table 4.9 lists the per-server X-engine status entries, where {n} is the 1-based server index.

Table 4.9: X-engine Server MIB Entries

MIB Entry	Type	Description
SERVER{n}_STAT	uint64	Server {n} status
SERVER{n}_INFO	char[256]	Server {n} information
SERVER{n}_SOFTWARE	char[256]	Software version
SERVER{n}_TEMP_MAX	float32	Maximum temperature
SERVER{n}_TEMP_MIN	float32	Minimum temperature
SERVER{n}_TEMP_AVG	float32	Average temperature
SERVER{n}_HOSTNAME	char[256]	Network hostname

Implementation Note

Server index 1 refers to the headnode. Indices 2–5 refer to the servers running the X-engine pipelines.

4.3.8 Global Status

Table 4.10 lists system-wide status entries.

4.3.9 Command Status

Table 4.11 lists the command status MIB entry.

The `CMD_STAT` structure contains:

Table 4.10: Global Status MIB Entries

MIB Entry	Type	Description
GLOBAL_TEMP_MAX	float32	Maximum system temperature
GLOBAL_TEMP_MIN	float32	Minimum system temperature
GLOBAL_TEMP_AVG	float32	Average system temperature
CLK_VAL	uint32	Milliseconds past midnight
UTC_START	char[256]	UTC start time string
UPTIME	uint32	System uptime in seconds

Table 4.11: Command Status MIB Entry

MIB Entry	Type	Description
CMD_STAT	struct	Command status for previous slot

- `slot_time` (uint32): Seconds past station time midnight
- `num_commands` (uint16): Number of commands ($N \leq 606$)
- `reference[N]` (uint32): Reference numbers
- `completion_code[N]` (uint8): Exit codes

4.4 Control Commands

4.4.1 PNG — Ping

The PNG command tests connectivity with the NDP subsystem (Table 4.12).

Table 4.12: PNG Command

Field	Value
Command	PNG
Data	(none)
Response	Status only

4.4.2 INI — Initialize

The INI command initializes the NDP subsystem (Table 4.13).

The initialization sequence:

1. Validate configuration file (pipeline and board counts)
2. Verify F-engine boards are accessible
3. Program FPGA firmware on all F-engine boards
4. Configure F-engine parameters (FFT shift, equalization)
5. Verify X-engine servers are accessible

Table 4.13: INI Command

Field	Value
Command	INI
Data	Mode string (optional flags)
Response	Status; on failure: exit code and message

6. Start X-engine pipelines
7. Verify data flow from F-engines to X-engines

Exit Codes

0x00	Process accepted without error
0x01	Invalid configuration file values
0x04	Board programming failed
0x05	Board configuration failed
0x08	Board-level shutdown failed
0x09	Server startup failed
0x0A	Server shutdown failed
0x0B	Pipeline startup failed
0x0C	Blocking operation in progress
0x0D	Internal F-engine error condition
0x0E	Pipeline processing error

Options

The INI command accepts the option strings listed in Table 4.14 (space-separated).

Table 4.14: INI Command Options

Option	Description
FORCE	Continue initialization even if errors occur on some boards
NOREPROGRAM	Skip FPGA programming (assume already programmed)
STARTFREQ_NNN	Set the starting frequency of the observed band to NNN MHz. The value is rounded to the nearest multiple of 16 channels (≈ 383 kHz). If not specified, the default starting frequency from the configuration file is used.

4.4.3 SHT — Shutdown

The SHT command shuts down the NDP subsystem (Table 4.15).

Table 4.15: SHT Command

Field	Value
Command	SHT
Data	Mode string (optional, space-separated)
Response	Status; on failure: exit code and message

Exit Codes

0x00	Process accepted without error
0x07	Invalid command arguments (unknown mode)
0x08	Board-level shutdown failed
0x0A	Server shutdown failed
0x0C	Blocking operation in progress

Options

The available options are listed in Table 4.16.

Table 4.16: SHT Command Options

Option	Description
SCRAM	Immediate hard shutdown without cleanup
RESTART	Restart system after shutdown
FORCE	Continue shutdown even if errors occur
HARD	Reboot F-engine boards instead of just unprogramming

4.4.4 STP — Stop

The STP command stops an active observing mode (Table 4.17).

Table 4.17: STP Command

Field	Value
Command	STP
Data	Mode to stop: “DRX”, “TBS”, “TBT”, “BEAM{n}”, “COR”
Response	Status; on failure: exit code and message

Exit Codes

0x00	Process accepted without error
0x63	Subsystem not ready
0xFF	Invalid mode string

Mode Strings

The valid mode strings are listed in Table 4.18.

Table 4.18: STP Mode Strings

Mode	Description
DRX	Stop all DRX/beam outputs
TBS	Stop TBS streaming mode
TBT	Stop TBT capture
BEAM1 – BEAM4	Stop specific beam output
COR	Stop correlator output

Implementation Note

The TBT and COR stop modes are currently no-ops. TBT captures complete on their own and COR runs continuously once the system is initialized.

4.4.5 DRX — Configure Beam Tuning

The DRX command configures a frequency tuning for a beam (Table 4.19). The command data format is given in Table 4.20.

Table 4.19: DRX Command

Field	Value
Command	DRX
Data	Binary packed structure (see below)
Response	Status; on failure: exit code and message

Table 4.20: DRX Command Data Format (Big-Endian)

Field	Type	Range	Description
beam	uint8	1–4	Beam number
tuning	uint8	1–2	Tuning number within beam
freq	float64	0–98 MHz	Center frequency in Hz
filter	uint8	1–7	Filter code (see Table 4.21)
gain	int16	0–15	Gain setting (right bit-shift)
high_dr	uint8	0–1	High dynamic range mode
subslot	uint8	0–99	Subslot for execution

Python struct format: `'>BBdBhBB'` (15 bytes total)

Exit Codes

0x00 Process accepted without error
 0x63 Subsystem not ready
 0xFF Invalid parameter

Table 4.21: DRX Filter Codes

Code	Sample Rate
1	250 kHz
2	500 kHz
3	1.0 MHz
4	2.0 MHz
5	4.9 MHz
6	9.8 MHz
7	19.6 MHz

4.4.6 TBS — Transient Buffer – Streaming

The **TBS** command configures the triggered buffer spectra mode (Table 4.22). The command data format is given in Table 4.23.

Table 4.22: TBS Command

Field	Value
Command	TBS
Data	Binary packed structure (see below)
Response	Status; on failure: exit code and message

Table 4.23: TBS Command Data Format (Big-Endian)

Field	Type	Range	Description
freq	float64	0–98 MHz	Center frequency in Hz
filter	uint8	7–9	Filter code (see Table 4.24)

Python struct format: `'>dB'` (9 bytes total)

Implementation Note

TBS configuration commands are sent to all X-engine pipelines but a pipeline will only output data if it handles the entire frequency range requested. Because of this any TBS requests that falls on a X-engine server/pipeline boundary will be accepted but not produce data.

4.4.7 TBT — Transient Buffer – Triggered

The **TBT** command configures and triggers a time-domain voltage capture (Table 4.25). The command data format is given in Table 4.26.

Python struct format: `'>Qiq'` (20 bytes total)

Table 4.24: TBS Filter Codes

Code	Sample Rate
7	≈ 95.7 kHz
8	≈ 191.4 kHz
9	≈ 287.1 kHz

Table 4.25: TBT Command

Field	Value
Command	TBT
Data	Binary packed structure (see below)
Response	Status; on failure: exit code and message

Implementation Note

If the mask value is negative, the absolute value is used as the mask and the data is written to local storage rather than transmitted over the network.

4.4.8 BAM — Beamformer Coefficients

The **BAM** command sets the beamformer delays and gains for a beam (Table 4.27). The command data format is given in Table 4.28.

The delays array contains one 16-bit value per input (512 total). The value is in fixed-point format with 12 integer bits and 4 fractional bits, representing the delay in samples.

The gains array contains a 2×2 polarization mixing matrix for each stand. The matrix layout is:

$$\begin{bmatrix} G_{XX} & G_{XY} \\ G_{YX} & G_{YY} \end{bmatrix}$$

Exit Codes

0x00 Process accepted without error
 0x63 Subsystem not ready
 0xFF Invalid parameter

Table 4.26: TBT Command Data Format (Big-Endian)

Field	Type	Range	Description
trigger	uint64	Full range	Trigger time in f_s ticks
samples	int32	Full range	Number of samples to capture
mask	int64	Full range	Stand mask (negative = local dump)

Table 4.27: BAM Command

Field	Value
Command	BAM
Data	Binary packed structure (see below)
Response	Status; on failure: exit code and message

Table 4.28: BAM Command Data Format (Big-Endian)

Field	Type	Range	Description
beam	uint16	1–4	Beam number
delays	uint16[512]	Full range	Delay per input (fixed 16.4)
gains	uint16[256][2][2]	Full range	Gain matrix per stand
subslot	uint8	0–99	Subslot for execution

4.4.9 COR — Correlator Configuration

The **COR** command configures the correlator (Table 4.29). The command data format is given in Table 4.30.

Table 4.29: COR Command

Field	Value
Command	COR
Data	Binary packed structure (see below)
Response	Status; on failure: exit code and message

Python struct format: `'>ihB'` (7 bytes total)

Exit Codes

- 0x00 Process accepted without error
- 0x63 Subsystem not ready
- 0xFF Invalid parameter

Implementation Note

The **COR** command is currently a no-op and the correlator output starts automatically with ≈ 5 s integrations once the system is initialized.

4.4.10 RPT — Report MIB Entry

The **RPT** command queries a MIB entry (Table 4.31).

4.5 System States

The NDP subsystem reports one of the status values listed in Table 4.32.

Table 4.30: COR Command Data Format (Big-Endian)

Field	Type	Range	Description
navg	int32	≥ 500	Integration time in subslots
gain	int16	0–15	Gain setting (right bit-shift)
subslot	uint8	0–99	Subslot for execution

Table 4.31: RPT Command

Field	Value
Command	RPT
Data	MIB entry name (ASCII string)
Response	MIB value; on failure: error message

Table 4.32: System Status Values

Status	Description
SHUTDWN	System is shut down or not yet initialized. No commands except INI will be processed.
BOOTING	System is in the process of initializing. Commands will be rejected with exit code 0x0C.
NORMAL	System is operating normally. All commands are accepted.
WARNING	System is operating but a warning condition exists. Commands are accepted.
ERROR	An error condition exists. The system may need to be re-initialized.

Section 5

Output Data Interfaces

5.1 Overview

The NDP subsystem produces several types of output data:

- Beamformer output (DRX) — phased array beam data, 4+4 bit complex
- Beamformer output (DRX8) — phased array beam data, 8+8 bit complex (high dynamic range)
- Correlator output (COR) — visibility data
- All dipoles output (TBS) — continuous frequency-domain spectra, 100% duty cycle
- All dipoles output (TBT) — triggered frequency-domain spectra dump, wider bandwidth

All output data is transmitted via UDP packets to data recorders or written to local storage.

The DRX and COR data products pass through the power line interference flagger before beamforming and correlation, respectively. TBS and TBT data are not flagged, preserving the raw channelized voltages for offline analysis. See Appendix B for details.

5.2 Beamformer Output (DRX/DRX8)

The beamformer produces time-domain complex samples for each beam and polarization. The T-engine converts the internal frequency-domain representation back to time domain for output.

5.2.1 Packet Header

Table 5.1 defines the DRX/DRX8 packet header fields.

Frame Count Word

The frame_count_word encodes beam, tuning, format, and polarization, as detailed in Table 5.2.

5.2.2 Packet Payload

Each packet payload contains 4096 complex samples from one polarization of one beam.

Table 5.1: DRX/DRX8 Packet Header (32 bytes)

Field	Type	Bytes	Offset	Description
sync_word	uint32	4	0	0x5CDECODE
frame_count_word	uint32	4	4	Frame count + ID (see below)
secs_count	uint32	4	8	Seconds since 1970-01-01 UTC
decimation	uint16	2	12	Sample rate decimation factor
time_offset	uint16	2	14	Time offset (Tnom) in f_s ticks
time_tag	uint64	8	16	Sample time in f_s ticks (big-endian)
tuning_word	uint32	4	24	Tuning frequency (big-endian)
flags	uint32	4	28	Status flags

Table 5.2: DRX Frame Count Word Bit Fields

Bits	Name	Description
0–2	beam	Beam number (0-based)
3–5	tuning	Tuning number (0-based)
6	drx8_flag	0 = DRX (4+4 bit), 1 = DRX8 (8+8 bit)
7	polarization	0 = X, 1 = Y
8–31	frame_count	Frame counter

DRX Format (4+4 bit)

Standard DRX packets use 4-bit real and 4-bit imaginary components packed into one byte per sample:

$$[4096 \text{ samples}][2 \text{ complex (I,Q)}][4 \text{ bits}] = 4096 \text{ bytes}$$

DRX8 Format (8+8 bit)

High dynamic range DRX8 packets use 8-bit real and 8-bit imaginary components:

$$[4096 \text{ samples}][2 \text{ complex (I,Q)}][8 \text{ bits}] = 8192 \text{ bytes}$$

The DRX8 format is selected by setting the `high_dr` flag in the DRX command.

5.2.3 Data Rate

For one 19.6 MHz dual-polarization beam:

$$\begin{aligned} \text{DRX: } & 2 \text{ tunings} * 2 \text{ pols} * 1 \text{ byte/sample} * 19.6\text{e6 samples/sec} = 78.4 \text{ MB/s} \\ \text{DRX8: } & 2 \text{ tunings} * 2 \text{ pols} * 2 \text{ bytes/sample} * 19.6\text{e6 samples/sec} = 156.8 \text{ MB/s} \end{aligned}$$

5.3 Correlator Output (COR)

The correlator produces visibility data for all baselines.

5.3.1 Packet Header

Table 5.3 defines the COR packet header fields.

Table 5.3: COR Packet Header

Field	Type	Bytes	Offset	Description
sync_word	uint32	4	0	0x5CDECODE
frame_count_word	uint32	4	4	Frame count + ID (see below)
secs_count	uint32	4	8	Seconds since 1970-01-01 UTC
freq_chan	int16	2	12	First channel in packet (0-based)
COR_GAIN	int16	2	14	Gain setting
time_tag	int64	8	16	Central time in f_s ticks
COR_NAVG	int32	4	24	Integration time in subslots
stand_i	int16	2	28	Unconjugated stand (1–256)
stand_j	int16	2	30	Conjugated stand (1–256)

Frame Count Word

The frame_count_word encodes server identification and format information, as detailed in Table 5.4.

Table 5.4: COR Frame Count Word Bit Fields

Bits	Name	Description
0–7	server_id	Server identifier
8–15	nserver	Number of servers
16–23	nchan_decim	Channel decimation
24–31	format_id	0x02 for COR

5.3.2 Packet Payload

Each packet contains N frequency channels and 4 polarization products for one baseline:

$$[N \text{ chans}] [2 \text{ pol}_i (X,Y)] [2 \text{ pol}_j (X,Y)] [8 \text{ bytes complex}] = 1056 \text{ bytes}$$

Polarizations are ordered X then Y, giving combined order: X_iX_j , X_iY_j , Y_iX_j , Y_iY_j , where ‘j’ elements represent the conjugated stand.

Each visibility value is stored as a little-endian complex64 (two 32-bit floats: real, imaginary).

Implementation Note

The COR output channels have a factor of four decimation relative to the input F-engine channels.

5.3.3 Data Rate

For 33-channel subbands with all baselines:

$$\begin{aligned}
 & 1056 \text{ bytes} * 256 * 257 / 2 \text{ baselines} / \text{COR_NAVG seconds} \\
 & = 34.7 \text{ MB} / \text{COR_NAVG} \\
 & = 3.47 \text{ MB/s} @ \text{COR_NAVG} = 10 \text{ seconds (1000 subslots)}
 \end{aligned}$$

5.4 TBX Format Overview

The TBS and TBT output modes use a unified packet format (TBX) derived from the legacy TBF format. Both modes output frequency-domain complex spectra for all inputs, but differ in their intended use case and channel configuration:

- **TBS** (Transient Buffer – Streaming): Continuous streaming mode designed for 100% duty cycle operation at narrow bandwidth (~ 200 kHz at filter code 8)
- **TBT** (Transient Buffer – Triggered): Triggered dump mode with wider bandwidth but reduced duty cycle, used for capturing transient events

5.4.1 TBX Packet Header

Table 5.5 defines the TBX packet header fields shared by TBS and TBT.

Table 5.5: TBX Packet Header (TBS/TBT) — 28 bytes

Field	Type	Bytes	Offset	Description
sync_word	uint32	4	0	0x5CDECODE
frame_count_word	uint32	4	4	Frame count + 0x08 in bits 24–31
secs_count	uint32	4	8	Seconds since 1970-01-01 UTC
first_chan	uint32	4	12	First channel (big-endian)
nstand	uint16	2	16	Number of stands
nchan	uint16	2	18	Number of channels
time_tag	uint64	8	20	Time tag in f_s ticks (big-endian)

5.4.2 TBX Packet Payload

Each packet contains frequency-domain data for multiple channels and all stands:

$$[N \text{ chans}][256 \text{ stands}][2 \text{ pols (X,Y)}][2 \text{ complex (I,Q)}][4 \text{ bits}] = N * 512 \text{ bytes}$$

The number of channels per packet varies by mode.

5.5 TBS Output (Transient Buffer – Streaming)

TBS is a continuous streaming mode that outputs frequency-domain spectra for all 512 inputs. It is designed for 100% duty cycle operation with a narrow bandwidth selection.

5.5.1 Configuration

Table 5.6 lists the TBS configuration parameters.

5.5.2 Data Rate

TBS is designed for sustainable continuous streaming. At filter code 8 with ~ 200 kHz bandwidth:

$$\sim 200 \text{ kHz} / 23.926 \text{ kHz/chan} = \sim 8 \text{ channels}$$

$$8 \text{ chans} * 512 \text{ inputs} * 1 \text{ byte/sample} * 23.926 \text{ kHz} = \sim 98 \text{ MB/s}$$

Table 5.6: TBS Configuration

Parameter	Value
Channels per packet	4, 8, or 12
Bandwidth (filter 8)	~200 kHz
Duty cycle	100% (continuous)
Data format	Complex 4+4 bit

5.6 TBT Output (Transient Buffer – Triggered)

TBT is a triggered dump mode that captures frequency-domain spectra with wider bandwidth than TBS. It trades duty cycle for increased bandwidth, making it suitable for capturing transient events.

5.6.1 Configuration

Table 5.7 lists the TBT configuration parameters.

Table 5.7: TBT Configuration

Parameter	Value
Channels per packet	16
Duty cycle	< 100% (triggered)
Data format	Complex 4+4 bit

5.6.2 Local Storage Option

When the TBT mask is specified as a negative value in the TBT command, the data is written to local storage on the X-engine servers rather than transmitted over the network. This allows for higher instantaneous data rates.

5.7 Output Summary

Table 5.8 provides an overview of all NDP output modes.

Table 5.8: NDP Output Summary

Mode	Format ID	Frame Size	Duty Cycle	Typical Rate
DRX	Bit 6 = 0	4128 bytes	100%	78.4 MB/s/beam
DRX8	Bit 6 = 1	8224 bytes	100%	156.8 MB/s/beam
COR	0x02 (bits 24–31)	Variable	100%	3.5 MB/s @ 10s
TBS	0x08 (bits 24–31)	Variable	100%	~98 MB/s
TBT	0x08 (bits 24–31)	Variable	<100%	≤100 MB/s when sent to the data recorder

5.8 Internal Interfaces

This section documents internal data formats that are available via multicast and may be accessed by other instruments or monitoring systems.

5.8.1 ZCU102 F-engine Output

The ZCU102 F-engine boards output channelized voltage data via multicast UDP packets. This data stream is the input to the X-engine servers but can also be received by other systems on the same network.

Packet Header

All header fields are big-endian (network byte order). Table 5.9 defines the header fields.

Table 5.9: ZCU102 Packet Header (32 bytes)

Field	Type	Bytes	Offset	Description
seq	uint64	8	0	Spectra/packet counter
sync_time	uint32	4	8	UNIX sync time (seconds)
npol	uint16	2	12	Polarizations in this packet
npol_tot	uint16	2	14	Total polarizations
nchan	uint16	2	16	Channels in this packet
nchan_tot	uint16	2	18	Total channels (for this pipeline)
chan_block_id	uint32	4	20	Channel block identifier
chan0	uint32	4	24	First channel in this packet
pol0	uint32	4	28	First polarization in this packet

Packet Payload

The payload contains channelized complex voltage data from the polyphase filter bank:

[nchan channels][npol polarizations][2 complex (I,Q)][4 bits]

Table 5.10 summarizes the payload format parameters.

Table 5.10: ZCU102 Payload Format

Parameter	Value
Data format	CI4 (4-bit I, 4-bit Q)
Sample size	1 byte per complex sample
Payload size	nchan × npol bytes
Channel bandwidth	≈23.926 kHz

Packet Addressing

Each ZCU102 board transmits data to the X-engine servers via multicast. The channel and polarization assignments are encoded in the header fields:

- **chan0**: Absolute first channel number in this packet

- `chan_block_id`: Block index for channel assignment
- `pol0`: First polarization index (related to the ZCU102 boards index within the system)
- The effective channel within a pipeline is: $\text{chan_block_id} \times \text{nchan}$

Source Identification

The source index for packet reassembly is computed as:

```

npol_blocks = npol_tot / npol
nchan_blocks = nchan_tot / nchan
nsrc = npol_blocks * nchan_blocks
src = (chan - chan0) / nchan * npol_blocks + (pol - pol0) / npol

```

Data Rate

Each ZCU102 board outputs channelized data for 32 inputs (16 stands $\times$ 2 polarizations):

$32 \text{ inputs} * 4096 \text{ channels} * 1 \text{ byte/sample} * 23.926 \text{ kHz} = \sim 3.1 \text{ GB/s per board}$

Total F-engine output (16 boards): $\sim 50 \text{ GB/s}$ aggregate

Site-Specific Configuration

The SNAP2 board at LWA-NA use the same packet header but include 64 inputs per packet instead of the ZCU102's 32 inputs.

Section 6

Safety Interface

6.1 Overview

The NDP subsystem includes several safety features to protect equipment from damage due to temperature anomalies or hardware failures.

6.2 Temperature Monitoring

The NDP continuously monitors temperature across all subsystems:

- F-engine board (ZCU102) FPGA die temperatures
- X-engine server temperatures (via IPMI sensors)
- Headnode temperatures

6.2.1 Temperature Sensors

F-engine Boards

Each ZCU102 board reports FPGA die temperature through the Xilinx system monitor interface.

Head Node and X-engine Servers

Server temperatures are monitored via IPMI sensors, which may include:

- CPU temperatures
- GPU temperatures
- Ambient temperatures
- Power supply temperatures

6.2.2 Temperature MIB Entries

The following MIB entries report temperature status. Table 6.1 lists the available temperature MIB entries.

Table 6.1: Temperature Monitoring MIB Entries

MIB Entry	Description
BOARD{n}_TEMP_MAX	Maximum temperature on F-engine board {n}
BOARD{n}_TEMP_MIN	Minimum temperature on F-engine board {n}
BOARD{n}_TEMP_AVG	Average temperature on F-engine board {n}
SERVER{n}_TEMP_MAX	Maximum temperature on server {n}
SERVER{n}_TEMP_MIN	Minimum temperature on server {n}
SERVER{n}_TEMP_AVG	Average temperature on server {n}
GLOBAL_TEMP_MAX	Maximum temperature across all systems
GLOBAL_TEMP_MIN	Minimum temperature across all systems
GLOBAL_TEMP_AVG	Average temperature across all systems

6.3 IPMI Power Control

The X-engine servers support out-of-band power control via IPMI. Table 6.2 lists the supported power operations.

Table 6.2: IPMI Power Operations

Operation	Description
status	Query current power state (on/off)
on	Power on server
off	Power off server
cycle	Power cycle server
reset	Reset server

6.4 Subsystem State Machine

The NDP subsystem operates in one of the states shown in Table 6.3.

Table 6.3: NDP State Machine

State	Description
SHUTDWN	System is powered down or not initialized. Only INI commands are processed.
BOOTING	System is initializing (programming FPGAs, starting pipelines). Commands are rejected with status 0x0C.
NORMAL	System is operating normally. All commands are accepted.
WARNING	A warning condition exists but the system continues to operate.
ERROR	An error condition exists. The system may need to be re-initialized.

6.4.1 State Transitions

SHUTDOWN	$\xrightarrow{\text{INI}}$	BOOTING
BOOTING	$\xrightarrow{\text{success}}$	NORMAL
BOOTING	$\xrightarrow{\text{failure}}$	ERROR
NORMAL	$\xrightarrow{\text{SHT}}$	SHUTDOWN
NORMAL	$\xrightarrow{\text{warning}}$	WARNING
NORMAL	$\xrightarrow{\text{error}}$	ERROR
WARNING	$\xrightarrow{\text{resolved}}$	NORMAL
WARNING	$\xrightarrow{\text{error}}$	ERROR
ERROR	$\xrightarrow{\text{SHT}}$	SHUTDOWN

6.5 F-engine Board Monitoring

The NDP monitors F-engine board health including:

- FPGA programming status
- FFT overflow counters
- Equalizer clip counts
- Ethernet transmit overflow/full status
- Data transmission status
- Input RMS levels

If an F-engine board fails to program or stops transmitting data, the system enters the **ERROR** state.

6.5.1 Input RMS Monitoring

In addition to the per-input statistics reported through the `ANT{n}_RMS` MIB entries, the NDP evaluates the distribution of input RMS values on each F-engine board to detect ADC problems. A board is considered reasonable when both of the following hold across the inputs on that board:

- The maximum RMS is less than 250 counts
- The standard deviation of the RMS values is less than 100 counts

The two criteria are intended to distinguish an analog signal path problem, where the gain is wrong across all inputs and the RMS values remain similar to one another, from an NDP problem, where only the inputs on one or more misbehaving ADCs depart from the rest.

Operational Note

This check is a heuristic and can mis-attribute the source of a problem. An analog signal path fault that affects only part of the array can trip the standard deviation criterion and be reported as an NDP condition, and conversely an ADC fault common to all of the inputs on a board can present as a uniform gain error. The reported condition should be treated as an indication that the input levels are wrong, not as a definitive statement of which subsystem is at fault. Confirm the source against the per-input `ANT{n}_RMS` values and the current ASP configuration before acting.

The check is evaluated once per monitoring poll. To avoid reacting to transient conditions, the system enters the **ERROR** state only after the check fails on three consecutive polls, and the condition is reported in the **SUMMARY** field of the MIB with exit code 0x0D:

Found N FPGA board(s) with unreasonable input RMS values

The hostnames of the affected boards are written to the system log.

6.6 X-engine Pipeline Monitoring

The NDP monitors X-engine pipeline status via ZMQ communication:

- Pipeline running status
- Data capture status
- Processing errors

6.7 Timing Monitor Monitoring

The NDP monitors the timing monitor board including:

- Power supply voltages (12V, 9V, 6V)
- Valon synthesizer lock status
- Recent detection of a sync pulse
- Microcontroller temperature

If the timing monitor board reports a problem, the system enters the **ERROR** state.

6.8 Error Recovery

To recover from most error conditions:

1. Identify the underlying cause via **INFO** and **LASTLOG** MIB entries
2. Address the hardware or configuration issue
3. Issue a **SHT** command if the system is not already shut down
4. Issue an **INI** command to re-initialize

Warning

If a blocking operation (**INI** or **SHT**) is in progress, additional **INI** or **SHT** commands will be rejected with exit code 0x0C (blocking operation in progress).

6.9 Command Validation

The NDP validates all commands before execution:

- Beam numbers must be in range 1–4
- Tuning numbers must be in range 1–2
- DRX filter codes must be in range 1–7; TBS filter codes must be in range 7–9
- Gain values must be in range 0–15
- Subslot values must be in range 0–99

Invalid commands are rejected with appropriate exit codes (see Appendix A).

Section 7

References

7.1 Reference Documents

1. MCS Common ICD — Monitor and Control System Common Interface Control Document
2. DP ICD — Digital Processor Interface Control Document (legacy system reference)
3. Draft ADP ICD — Draft version of the Advanced Digital Processor Interface Control Document
4. ASP ICD — Analog Signal Processor Interface Control Document
5. LWA Station Architecture Document
6. J. Sievers, “Inverting a Polyphase Filter Bank,” unpublished note, 2022. Distributed via the CASPER mailing list. Reference implementation at <https://github.com/dcxSt/pfb-mod>.
7. J. W. Romein, “An Efficient GPU-Based Correlator X-Engine Implemented Using Tensor Cores,” *Astronomy & Astrophysics*, vol. 656, A52, 2021. DOI: 10.1051/0004-6361/202141896.

7.2 Hardware References

- Xilinx ZCU102 Evaluation Kit User Guide (UG1182)
- Xilinx Zynq UltraScale+ MPSoC Data Sheet (DS891)

7.3 Software Components

The following software components implement the NDP system. Table 7.1 lists the key modules.

7.4 Pipeline Blocks

The X-engine and T-engine processing pipelines are implemented as sequences of processing blocks defined in `ndp_drx.py` and `ndp_tengine.py`. Table 7.2 lists the X-engine blocks and Table 7.3 lists the T-engine blocks. Figures 7.1 and 7.2 show the corresponding Bifrost dataflow graphs, including the ring buffer data types between stages and CPU core assignments.

Table 7.1: NDP Software Modules

Module	Description
ndp_control.py	Main NDP control daemon; MCS command interface
ndp/Ndp.py	Core NDP functions; command processing
ndp/NdpCommon.py	Constants and fixed parameters
ndp/MCS2.py	MCS message protocol implementation
ndp_drx.py	X-engine beamformer and correlator pipeline
ndp_tengine.py	T-engine pipeline (frequency to time domain)
lwa_f	F-engine control library (ZCU102 and SNAP2)

Table 7.2: X-engine Pipeline Blocks (ndp_drx.py)

Block	Description
CaptureOp	F-engine packet capture and assembly
BufferCopyOp	Ring buffer copy between pipeline stages
GPUCopyOp	Host-to-GPU data transfer
PowerLineFlaggerOp	Power line interference flagging (Appendix B)
BeamformerOp	GPU beamformer (delay, gain, and sum)
CorrelatorOp	GPU correlator (full visibility matrix; Appendix C)
TriggeredDumpOp	TBT triggered spectra dump
StreamingOp	TBS continuous spectra streaming
RetransmitOp	Beamformed data retransmission to T-engines
PacketizeOp	COR output packetization

Table 7.3: T-engine Pipeline Blocks (ndp_tengine.py)

Block	Description
CaptureOp	Beamformed data capture from X-engines
GPUCopyOp	Host-to-GPU data transfer
ReChannelizerOp	Inverse polyphase filter bank (Appendix D)
TEngineOp	Frequency-to-time domain conversion
PacketizeOp	DRX/DRX8 output packetization

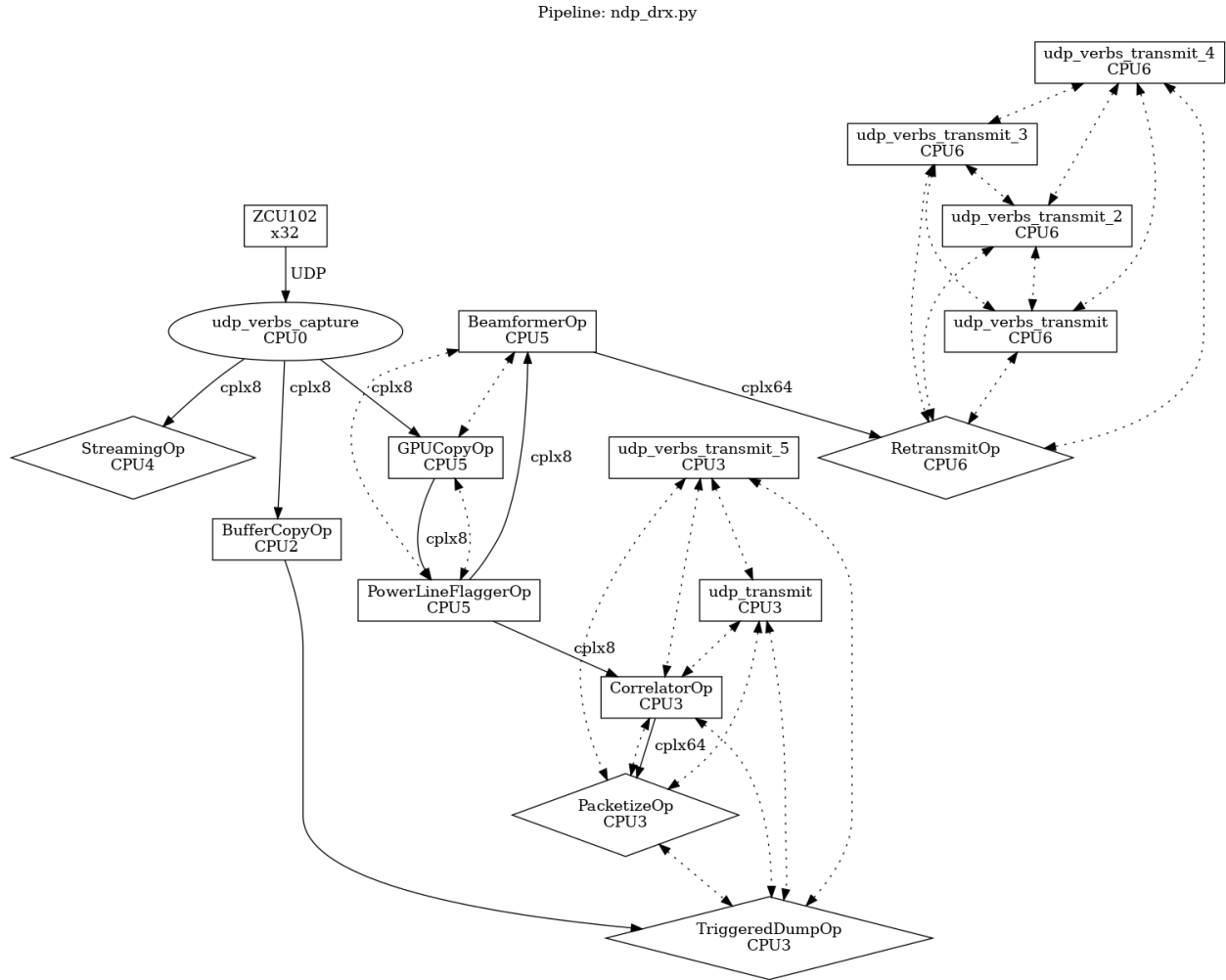


Figure 7.1: X-engine Bifrost pipeline dataflow graph. Solid lines indicate ring buffer connections with data types labeled (cplx8/cplx64). Dotted lines indicate sequence callback connections. Node shapes follow Bifrost conventions: ellipses for capture blocks, rectangles for processing blocks, and diamonds for output/transmit blocks.

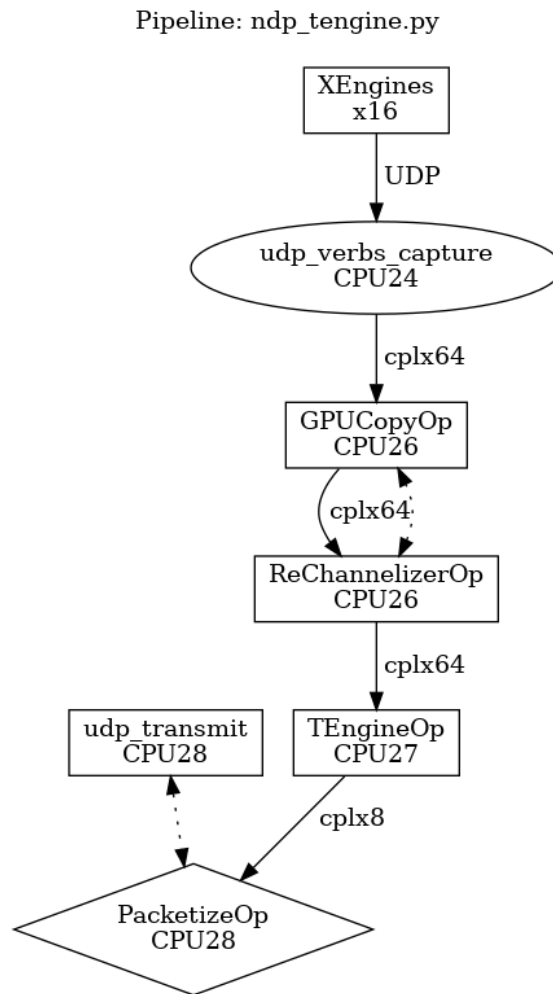


Figure 7.2: T-engine Bifrost pipeline dataflow graph. The pipeline is primarily linear, converting frequency-domain beamformed data (cplx64) to time-domain output (cplx8) via the inverse polyphase filter bank and T-engine blocks.

7.5 Configuration

NDP configuration is stored in JSON format in site-specific files (`ndp_config.json.site`). The top-level keys and their nested structure are described below.

7.5.1 Global Parameters

Three top-level scalar parameters control NDP daemon timing (Table 7.4).

Table 7.4: Top-level Configuration Parameters

Key	Type	Description
<code>shutdown_timeout</code>	float	Graceful shutdown timeout (seconds)
<code>monitor_interval</code>	float	System monitoring interval (seconds)
<code>failsafe_interval</code>	float	Failsafe check interval (seconds)

7.5.2 mcs — MCS Communication

The `mcs` object configures the UDP interface between NDP and MCS, split into headnode and server sub-objects (Table 7.5).

Table 7.5: `mcs` Configuration Parameters

Key	Type	Description
<i>mcs.headnode</i>		
<code>local_host</code>	string	Bind address for receiving MCS commands
<code>local_port</code>	int	UDP port for receiving MCS commands
<code>remote_host</code>	string	MCS host address for sending responses
<code>remote_host_TEST</code>	string	Alternate MCS host for testing
<code>remote_port</code>	int	UDP port for sending MCS responses
<i>mcs.server</i>		
<code>local_host</code>	string	Bind address for server-side MCS interface
<code>local_port</code>	int	UDP port for server-side MCS interface

7.5.3 host — System Hostnames

The `host` object maps logical roles to network hostnames (Table 7.6).

7.5.4 ipmi — IPMI Credentials

The `ipmi` object provides credentials for out-of-band server management (Table 7.7).

7.5.5 fpga — F-engine Configuration

The `fpga` object configures the F-engine boards, including FPGA firmware, filter bank settings, and temperature thresholds (Table 7.8).

Table 7.6: **host** Configuration Parameters

Key	Type	Description
servers	array of string	X-engine server hostnames
fpgas	array of string	F-engine board hostnames
servers-data	array of string	Data network interface hostnames (two per server)
tengines	array of string	T-engine host identifiers

Table 7.7: **ipmi** Configuration Parameters

Key	Type	Description
username	string	IPMI username
password	string	IPMI password

Table 7.8: **fpga** Configuration Parameters

Key	Type	Description
username	string	SSH username for F-engine boards (ZCU102 only)
password	string	SSH password for F-engine boards (ZCU102 only)
data_ip_base	string	Base IP address for F-engine data interfaces
data_port_base	int	Base UDP port for F-engine data output
firmware	string	Path to FPGA firmware file (.fpg)
max_program_attempts	int	Maximum FPGA programming retries
fft_shift	int	FFT shift schedule for the polyphase filter bank
equalizer_coeffs	string	Path to equalizer coefficient file
bypass_pfb	bool	Bypass the polyphase filter bank
nchan_packet	int	Number of frequency channels per output packet
temperatures	array of string	IPMI temperature sensor names
temperature_warning	float	Warning temperature threshold (°C)
temperature_shutdown	float	Shutdown temperature threshold (°C)
temperature_scram	float	Emergency shutdown threshold (°C)

7.5.6 server — X-engine Server Configuration

The **server** object configures the X-engine GPU servers, including data network interfaces, resource monitoring, and temperature thresholds (Table 7.9).

Table 7.9: **server** Configuration Parameters

Key	Type	Description
username	string	SSH/management username
password	string	SSH/management password
cpu_ids	array of int	CPU socket IDs to monitor
gpu_ids	array of int	GPU device IDs to monitor
disk_ids	array of string	Disk mount points to monitor
data_ports	array of int	UDP data reception ports (one per server)
data_ifaces	array of string	Network interface names for data reception
startup_timeout	int	Server startup timeout (seconds)
temperatures	array of string	IPMI temperature sensor names
temperature_warning	float	Warning temperature threshold (°C)
temperature_shutdown	float	Shutdown temperature threshold (°C)
temperature_scram	float	Emergency shutdown threshold (°C)

7.5.7 drx — X-engine Pipeline Configuration

The **drx** key is an array of objects, one per X-engine pipeline instance. Each pipeline processes a contiguous range of frequency channels (Table 7.10).

Table 7.10: **drx[i]** Configuration Parameters

Key	Type	Description
first_channel	int	First frequency channel index processed
capture_bandwidth	float	Capture bandwidth (Hz)
beam_count	int	Number of beams produced
pipeline_idx	int	Pipeline index on the host server
tengine_idx	array of int	T-engine indices receiving beamformed data
tbx_recorder_idx	int	Recorder index for TBT/TBS output
cor_recorder_idx	int	Recorder index for correlator output
cpus	array of int	CPU core IDs allocated to this pipeline
gpus	array of int	GPU device IDs allocated to this pipeline
powerline_flagger	bool	Enable power line interference flagging

7.5.8 tengine — T-engine Pipeline Configuration

The **tengine** key is an array of objects, one per T-engine pipeline instance (Table 7.11).

Table 7.11: `tengine[i]` Configuration Parameters

Key	Type	Description
<code>pipeline_idx</code>	int	Pipeline index on the host server
<code>recorder_idx</code>	int	Recorder index for DRX output
<code>cpus</code>	array of int	CPU core IDs allocated to this pipeline
<code>gpus</code>	array of int	GPU device IDs allocated to this pipeline
<code>pfb_inverter</code>	bool	Enable inverse polyphase filter bank for DRX output

7.5.9 recorder — Data Recorder Configuration

The `recorder` key is an array of objects, one per data recorder. Recorders are referenced by index from the `drx` and `tengine` pipeline configurations (Table 7.12).

Table 7.12: `recorder[i]` Configuration Parameters

Key	Type	Description
<code>host</code>	string	Recorder hostname or IP address
<code>port</code>	int	UDP receive port on the recorder
<code>max_bytes_per_sec</code>	int	Maximum output data rate (bytes/sec)

7.5.10 tbt — Transient Buffer Triggered Configuration

The `tbt` object controls the transient buffer triggered capture ring buffer (Table 7.13).

Table 7.13: `tbt` Configuration Parameters

Key	Type	Description
<code>buffer_time_sec</code>	float	TBT ring buffer duration (seconds)

7.5.11 log — Logging Configuration

The `log` object controls log file rotation, formatting, and output paths (Table 7.14).

7.6 Source Code Repositories

- NDP Control Software: `ng_digital_processor`
- F-engine Control: `lwa_f`

Table 7.14: log Configuration Parameters

Key	Type	Description
<code>days_per_file</code>	int	Log file rotation period (days)
<code>max_file_count</code>	int	Maximum number of retained log files
<code>msg_format</code>	string	Python <code>logging</code> message format
<code>stats_format</code>	string	Statistics log message format
<code>date_format</code>	string	Date format string for log entries
<i>log.files</i>		
<code>server_temps</code>	string	Path to server temperature log file
<code>roach_temps</code>	string	Path to F-engine temperature log file

Appendix A

Exit and Status Codes

A.1 Command Exit Codes

Table A.1 lists all exit codes returned in response to control commands. Per-command exit codes are documented alongside each command in Section 4.4.

Table A.1: Command Exit Codes

Code	Description
0x00	Process accepted without error
0x01	Invalid configuration file values
0x04	Board programming failed
0x05	Board configuration failed
0x07	Invalid command arguments (unknown mode)
0x08	Board-level shutdown failed
0x09	Server startup failed
0x0A	Server shutdown failed
0x0B	Pipeline startup failed
0x0C	Blocking operation in progress (INI or SHT already running)
0x0D	Internal F-engine error condition
0x0E	Pipeline processing error
0x0F	Other internal error (e.g., timing monitor problem)
0x63	Subsystem not ready (in SHUTDOWN or BOOTING state)
0xFF	Unknown command or unimplemented feature

A.2 Subsystem Status Codes

See Table 4.32 in Section 4 for the status values reported in the SUMMARY field of MCS responses.

Appendix B

Power Line Interference Flagger

B.1 Background

Radio frequency interference (RFI) from overhead power lines is a persistent concern for the LWA. The dominant mechanism is *micro arcing*: small electrical discharges that occur at hardware junctions on power line infrastructure (e.g., insulators, connectors, and transformers). Each micro arc produces a short, broadband burst of electromagnetic radiation. Because the arcing is driven by the 60 Hz AC mains cycle, these bursts repeat at 60 Hz and its harmonics, leading to a characteristic “picket fence” pattern in the time domain with a period of approximately 16.7 ms.

In the frequency-channelized data produced by the NDP F-engines, a micro arc event appears as a transient increase in power across many or all frequency channels simultaneously for one or a few consecutive spectra. Because the bursts are broadband (spanning the full LWA band), they are visible at all frequencies and cannot be excised by simple channel blanking. Instead, the NDP uses a real-time temporal flagging algorithm, the **PowerLineFlaggerOp**, to detect and replace contaminated time samples before they are integrated into the beamformed and correlated data products.

B.2 Scope of Flagging

The power line flagger operates in the X-engine pipeline and processes data *after* the F-engine channelizer output has been assembled but *before* the beamformer and correlator. This means that the following data products are protected by the flagger:

- **DRX** (beamformed output) — flagged data are replaced before beamforming
- **COR** (correlator output) — flagged data are replaced before correlation

The following data products are *not* affected by the power line flagger:

- **TBT** (triggered buffer) — operates on a separate data path after the flagger, capturing raw channelized data
- **TBS** (streaming buffer) — operates on a separate data path after the flagger, streaming raw channelized data

This design is intentional: TBT and TBS are meant to provide unmodified voltage-level data for offline analysis, where users may wish to apply their own RFI mitigation strategies.

B.3 Algorithm Overview

The flagger processes data in “gulps” of 2500 spectra. At the NDP’s native time resolution of one spectrum every $8192/196 \times 10^6 \approx 41.8 \mu\text{s}$, each gulp spans approximately 104.5 ms, or roughly 6.3 periods of the 60 Hz mains cycle.

The algorithm proceeds in three stages:

B.3.1 Stage 1: Mean Power Computation

For each time sample in the gulp, the flagger computes the mean power across *all* stands and polarizations in groups of 16 frequency channels (the “channel block size”). Specifically, for each of the $4096/16 = 256$ channel blocks and each time sample, the algorithm:

1. Unpacks the 4-bit complex (CI4) samples for all 512 inputs ($256 \text{ stands} \times 2 \text{ polarizations}$) in the 16-channel block
2. Computes $|s|^2 = \text{Re}(s)^2 + \text{Im}(s)^2$ for each sample
3. Averages these power values to produce a single mean power value per channel block per time sample

This averaging across all inputs suppresses input-specific fluctuations and enhances the visibility of array-wide transient events such as power line arcs.

B.3.2 Stage 2: Baseline Estimation and Burst Detection

The 2500 time samples in each gulp are divided into overlapping windows at two timescales, as shown in Table B.1.

Table B.1: Flagger Time Window Parameters

Window	Size (spectra)	Duration	Purpose
Fast	16	$\sim 669 \mu\text{s}$	Burst detection
Slow	32	$\sim 1.34 \text{ ms}$	Baseline estimation

The “fast” windows are formed by averaging consecutive groups of 16 time samples. The “slow” windows are formed by averaging pairs of adjacent fast windows, producing windows that overlap by 16 samples with a stride of 16 samples.

The baseline noise level is estimated from the slow window with the *minimum* mean power. Within this quietest window, the algorithm computes both the mean power ($\bar{P}$) and the standard deviation (σ_P) of the per-sample power values. The minimum-power window is chosen because it is least likely to contain a burst event, providing a clean estimate of the quiescent noise floor.

B.3.3 Stage 3: Flagging and Replacement

Each individual time sample is compared against a threshold:

$$P_{\text{threshold}} = \bar{P} + N_{\sigma} \cdot \sigma_P \quad (\text{B.1})$$

where N_{σ} is a configurable parameter (default: 8). Any time sample whose mean power (in a given channel block) exceeds this threshold is flagged.

For each flagged time sample, all 16 channels in the affected channel block are replaced with appropriately-scaled Gaussian noise. The noise amplitude σ is derived from the quiescent mean power via the relation for the exponential distribution of power in Gaussian noise ($\sigma_{\text{raw}} = \sqrt{\bar{P}/2}$), with a quantization correction that accounts for the effect of clipping on the relationship between the expectation value of the exponential distribution and the underlying Gaussian standard deviation. For unclipped data, $\langle |z|^2 \rangle = 2\sigma^2$, but after clipping to the 4-bit range (± 7) the observed mean power underestimates the true σ . The correction is:

$$\sigma_{\text{repl}} = \sigma_{\text{raw}} \times (a_3 \sigma_{\text{raw}}^3 + a_2 \sigma_{\text{raw}}^2 + a_1 \sigma_{\text{raw}} + a_0) \quad (\text{B.2})$$

with the polynomial coefficients given in Table B.2.

Table B.2: Quantization Correction Polynomial Coefficients

Coefficient	Value
a_3	$1.15267220 \times 10^{-5}$
a_2	$-8.93112370 \times 10^{-4}$
a_1	$2.49709188 \times 10^{-2}$
a_0	$8.55697984 \times 10^{-1}$

These coefficients are derived from a Monte Carlo simulation of the quantization process. For complex Gaussian noise $z = x + iy$ where $x, y \sim \mathcal{N}(0, \sigma)$, the squared magnitude $|z|^2$ follows an exponential distribution with mean $\langle |z|^2 \rangle = 2\sigma^2$, so that $\sigma = \sqrt{\langle |z|^2 \rangle / 2}$. After clipping to the CI4 range (± 7), the observed mean power $\langle |z_q|^2 \rangle < 2\sigma^2$ because clipping suppresses the tails of the Gaussian distribution, and the naïve estimate $\sigma_{\text{raw}} = \sqrt{\langle |z_q|^2 \rangle / 2}$ underestimates σ . The correction factor $c(\sigma_{\text{raw}}) = \sigma_{\text{true}} / \sigma_{\text{raw}}$ is computed by simulating 5×10^5 complex samples at each of 101 values of σ spanning 0.5 to 20, quantizing and clipping to ± 7 , and fitting a third-order polynomial to the resulting correction curve as a function of σ_{raw} .

The replacement noise is drawn from a pre-generated pool of 2^{28} random samples stored in GPU memory to avoid the overhead of per-sample random number generation.

Design Note

The replaced samples are constrained to the 4-bit range (-7 to $+7$) to match the CI4 data format. The quantization correction ensures that the replaced noise, after clipping, has the same power statistics as the quiescent signal.

B.4 Timing Considerations

Table B.3 summarizes the key timing parameters of the power line flagger.

Table B.3: Power Line Flagger Timing

Parameter	Value
Spectrum period	$8192/196 \times 10^6 \approx 41.8 \mu\text{s}$
Gulp size	2500 spectra ($\sim 104.5 \text{ ms}$)
Fast window	16 spectra ($\sim 669 \mu\text{s}$)
Slow window	32 spectra ($\sim 1.34 \text{ ms}$)
Fast windows per gulp	156
Slow windows per gulp	155
60 Hz period	$\sim 16.67 \text{ ms}$ ($\sim 399 \text{ spectra}$)
60 Hz cycles per gulp	~ 6.3

Since a typical micro arc burst lasts for less than a millisecond, the fast window of $\sim 669 \mu\text{s}$ is well-matched to capturing individual events without excessive dilution from quiescent data. The slow window of $\sim 1.34 \text{ ms}$ encompasses roughly two burst durations, ensuring a robust baseline even if one sub-window is partially contaminated.

B.5 Monitoring and Reporting

B.5.1 Statistics Tracking

The flagger maintains two flagging statistics that are updated after every gulp:

- **Instantaneous flag fraction** (`flag_frac_now`): The fraction of data samples flagged in the most recent gulp, computed as the total number of flagged channel-block/time entries divided by the total data volume.
- **Moving average flag fraction** (`flag_frac_move`): An exponential moving average of the instantaneous flag fraction with a smoothing factor of $\alpha = 0.0005$:

$$F_{\text{move}}^{(n)} = \alpha \cdot F_{\text{now}}^{(n)} + (1 - \alpha) \cdot F_{\text{move}}^{(n-1)} \quad (\text{B.3})$$

At the gulp rate of $\sim 9.6 \text{ gulps/second}$, the e -folding time of this moving average is approximately 3.5 minutes.

Both statistics are published to the Bifrost process log system and are available to the pipeline monitor.

B.5.2 Log Messages

The flagger logs the moving average flag fraction every 1500 gulps, which corresponds to approximately every 2.6 minutes. These messages appear at the `INFO` level and have the format:

```
PowerLineFlagger - flagged X.XX%
```

B.5.3 System Health Integration

The NDP pipeline monitor reads the flagging statistics from each X-engine pipeline. If the moving average flag fraction exceeds 2% on any pipeline, the system status is elevated to `WARNING` and the condition is reported in the `SUMMARY` field of the MIB:

`<host>, DRX-<n> -- Flagging fraction at X.X%`

This threshold provides an early indication that power line interference is significant enough to potentially degrade data quality, even after flagging and replacement.

Operational Note

A sustained flag fraction above 2% does not mean that the data are unusable—the flagged samples have been replaced with appropriately-scaled noise. However, it does mean that a non-trivial fraction of the data has been substituted, which reduces the effective integration time and may affect the noise level of the beamformed and correlated outputs.

Appendix C

GPU Correlation with the Tensor Core Correlator

C.1 Overview

The NDP X-engine correlator computes the full visibility matrix for all 512 inputs (256 stands $\times$ 2 polarizations) in real time. Rather than implementing a custom GPU correlation kernel, NDP uses the *Tensor Core Correlator* (TCC), a high-performance correlation library developed by ASTRON that exploits the tensor core units available on modern NVIDIA GPUs (Reference 7). A Bifrost wrapper library, `btcc`, integrates the TCC into the Bifrost pipeline framework used by the X-engines.

C.2 Tensor Core Correlator

The TCC performs the computationally intensive outer-product accumulation that forms the core of an FX correlator’s X-engine. It accepts channelized voltage data for all inputs and produces cross-correlation products (visibilities) for all baselines, including auto-correlations. For N inputs the number of unique baselines (including autos) is $N(N + 1)/2$; with two polarizations the correlator produces four polarization products (XX, YY, XY, YX) per baseline.

The TCC achieves high throughput by mapping the correlation operation onto NVIDIA tensor core matrix-multiply-accumulate instructions. This mapping imposes a constraint on the time axis: the number of time samples per call must be a multiple of a quantity called $n_{\text{time_per_block}}$, defined as

$$n_{\text{time_per_block}} = \frac{128}{n_{\text{bits}}} \quad (\text{C.1})$$

where n_{bits} is the number of bits per component of the complex input samples. Table C.1 lists the supported bit depths and corresponding time constraints.

C.3 Bifrost Wrapper

The Bifrost wrapper, `btcc`, provides a Python/C++ interface that allows the TCC to be used as a processing block within a Bifrost pipeline. The wrapper is implemented as a Bifrost plugin and

Table C.1: TCC Time-Axis Constraints by Input Bit Depth

Input Type	n_{bits}	$n_{\text{time_per_block}}$
CI4 (4-bit complex)	4	32
CI8 (8-bit complex)	8	16
CF16 (16-bit complex)	16	8

handles three responsibilities:

- **Input reordering (swizzle).** The TCC expects input data in a specific memory layout dictated by the tensor core matrix dimensions. The wrapper includes a GPU kernel (`swizzle_kernel`) that transposes the incoming data from the Bifrost ring buffer layout into the layout required by the TCC.
- **Accumulation.** The TCC produces one correlation result per call. The wrapper maintains a GPU-resident accumulation buffer and adds each TCC output to the running sum.
- **Output reordering (deswizzle).** When the pipeline signals a dump (at the end of an integration period), a second GPU kernel (`reorder_kernel`) copies the accumulated visibilities from the TCC’s internal baseline ordering to the standard upper-triangle ordering expected by the rest of the pipeline, applying the appropriate conjugation. The accumulator is then reset.
- **CUDA graph optimization.** After the first execution, the wrapper captures the sequence of GPU operations (swizzle, correlate, accumulate) into a CUDA graph. Subsequent calls replay the captured graph, reducing kernel launch overhead.

The wrapper exposes three operations:

`init( $n_{\text{bits}}$ ,  $n_{\text{time}}$ ,  $n_{\text{chan}}$ ,  $n_{\text{stand}}$ ,  $n_{\text{pol}}$ ,  $n_{\text{decim}}$ )` Initialize the TCC and allocate intermediate buffers.

`execute(input, output, dump)` Correlate one gulp of data and accumulate. If `dump` is true, flush the accumulator to the output and reset.

`reset_state()` Zero the accumulation buffer.

Table C.2 summarizes the input and output array shapes.

Table C.2: Bifrost TCC Wrapper Array Shapes

Array	Shape	Type	Space
Input	$(n_{\text{time}}, n_{\text{chan}}, n_{\text{stand}} \cdot n_{\text{pol}})$	CI8	CUDA
Output	$(n_{\text{chan}}/n_{\text{decim}}, n_{\text{baseline}} \cdot n_{\text{pol}}^2)$	CI32	CUDA

where $n_{\text{baseline}} = n_{\text{stand}}(n_{\text{stand}} + 1)/2$.

C.4 NDP Integration

C.4.1 Time-Axis Padding

The `CorrelatorOp` block in the X-engine pipeline (`ndp_drx.py`) uses the `btcc` wrapper to correlate each gulp of channelized data. Because NDP operates the correlator with 8-bit complex (CI8) input data, the TCC requires n_{time} to be a multiple of 16. The pipeline gulp size of 512 spectra satisfies this constraint directly. In general, if a gulp size is not a multiple of 16, the `CorrelatorOp` pads n_{time} up to the next multiple of 16 at initialization:

$$n_{\text{time_padded}} = \text{ceil}(n_{\text{time_gulp}} / 16) * 16$$

Any extra time samples are zero-filled in the intermediate input array and contribute negligibly to the accumulated visibilities.

C.4.2 Channel Decimation

Each X-engine pipeline processes a subset of the total frequency channels from the F-engine channelizer; the number of channels per pipeline is set by the site configuration file. To reduce both the computational cost of correlation and the output data rate, the `CorrelatorOp` applies a frequency decimation factor of 4 *before* invoking the TCC. This decimation is implemented as a `BFMap` operation that unpacks the CI4 input samples, sums groups of 4 adjacent frequency channels, and writes the result as CI8 data into the intermediate input array. The TCC then operates on the decimated data with no knowledge of the original channel count.

The sum of 4 CI4 values (each in the range ± 7) has a maximum magnitude of $4 \times 7 = 28$, well within the CI8 range of ± 127 . Larger decimation factors would risk overflow into the CI8 representation and would require either wider intermediate storage or rescaling.

Table C.3 shows an example channel budget for the default configuration.

Table C.3: Correlator Channel Budget (Default Configuration)		
Stage	Per Pipeline	Total (16 Pipelines)
F-engine output	256	4096
After decimation ($\times 4$)	64	1024

The decimated channel bandwidth is $4 \times f_c \approx 95.7$ kHz.

C.4.3 Correlation and Accumulation

The correlation and accumulation cycle proceeds as follows:

1. Each gulp of $n_{\text{time_gulp}}$ spectra is copied into the padded intermediate array.
2. `bfcc.execute()` is called with `dump=0` for intermediate gulps and `dump=1` for the final gulp in an integration period.
3. When a dump occurs, the accumulated visibilities are reordered from the TCC's internal baseline ordering to the NDP output ordering (per-baseline, per-channel) and packetized by the downstream `PacketizeOp`.

The integration time is set by the `COR` command's `navg` parameter, which specifies the total number of spectra to integrate. The `CorrelatorOp` accumulates across multiple gulps until `navg` spectra have been processed, then triggers a dump.

Appendix D

PFB Inversion in the T-Engine

D.1 Motivation

The NDP F-engines channelize the incoming voltage data using a polyphase filter bank (PFB) with $n_{\text{tap}} = 4$ taps and $n_{\text{chan}} = 4096$ channels, producing a channel bandwidth of $f_s/2n_{\text{chan}} \approx 23.9$ kHz. The T-engine (beamformer) must synthesize a wideband time series from these channelized data before applying delays and re-channelizing to the output frequency resolution. A naïve approach—simply performing an inverse FFT on the channelized data—ignores the windowing applied by the PFB and introduces spectral artifacts at channel boundaries. To recover a faithful time series, the T-engine implements a PFB inversion that undoes the effects of both the Fourier transform and the PFB window function.

D.2 Approach

The PFB inversion follows the method described by Sievers¹, which formulates the forward PFB as a matrix equation

$$\mathbf{d}_{\text{PFB}} = \mathbf{F} \mathbf{S} \mathbf{W} \mathbf{d} \quad (\text{D.1})$$

where $\mathbf{d}$ is the raw time series, $\mathbf{W}$ is a diagonal matrix applying the windowed-sinc tap weights, $\mathbf{S}$ is a stacking matrix that folds the windowed data into n_{tap} segments and sums them, and $\mathbf{F}$ is the DFT. The inverse DFT undoes $\mathbf{F}$, leaving the product $\mathbf{S} \mathbf{W}$ to be inverted. Because $\mathbf{S} \mathbf{W}$ only couples samples separated by $2n_{\text{chan}}$, the problem decomposes into $2n_{\text{chan}}$ independent sub-problems, each involving a band-diagonal Toeplitz matrix of size n_{tap} .

Imposing circulant boundary conditions on these sub-problems allows the pseudo-inverse to be computed efficiently via FFTs: the inverse is applied by transforming along the tap axis, dividing by the eigenvalues of the circulant matrix (which are just the DFT of the sub-sampled window function), and transforming back. Because quantization in the F-engine introduces noise that is amplified by small eigenvalues, a Wiener filter is applied in the eigenvalue domain to regularize the inversion.

¹J. Sievers, “Inverting a Polyphase Filter Bank,” unpublished note, 2022. See also Reference 6.

D.3 NDP Implementation

The T-engine's `ReChannelizerOp` implements the PFB inversion on the GPU as follows:

1. **Window construction.** The PFB window is a 4-tap Hamming-windowed sinc, matching the F-engine firmware:

$$w[n] = \text{hamming}(4 \cdot n_{\text{chan}}) \times \text{sinc}\left(\frac{n}{n_{\text{chan}}}\right) \quad (\text{D.2})$$

This window is reshaped into a $4 \times n_{\text{chan}}$ matrix corresponding to the n_{tap} sub-sampled phases.

2. **Eigenvalue computation.** The sub-sampled window is embedded in a zero-padded matrix and Fourier-transformed along the tap axis to obtain the eigenvalues $H[k]$ of the circulant sub-problems.
3. **Wiener-filtered pseudo-inverse.** The inverse filter is constructed as

$$G[k] = \frac{|H[k]|^2}{|H[k]|^2 + \epsilon^2} \cdot \frac{1 + \epsilon^2}{H^*[k]} \quad (\text{D.3})$$

where $\epsilon = 0.3$ is the Wiener regularization threshold. This suppresses noise amplification at eigenvalues where the forward PFB strongly attenuates signal power.

4. **Inversion.** For each gulp of channelized data:
 - (a) An inverse FFT across frequency channels recovers the (windowed) time series.
 - (b) A 4-point DFT along the tap axis, pointwise multiplication by $G[k]$, and a 4-point inverse DFT apply the pseudo-inverse. The 4-point transforms are implemented as hand-unrolled butterfly operations for efficiency.
 - (c) A forward FFT re-channelizes the recovered time series to the output frequency resolution.

The PFB inverter is enabled by default in the T-engine configuration and can be toggled via the `pfb_inverter` key in the T-engine configuration block.

Design Note

The Wiener threshold of $\epsilon = 0.3$ was chosen empirically to balance reconstruction fidelity against quantization noise amplification for the NDP's 4-bit (CI4) channelized data. The $(1 + \epsilon^2)$ normalization in Equation D.3 preserves the overall signal amplitude so that the reconstructed time series has the correct power level.

D.4 Effect on Output Data

When the PFB inverter is enabled, the beamformed output gains are adjusted by -3 counts (in the DRX gain field) relative to the non-inverted case to account for the change in signal normalization. The inversion also removes the scalloping loss inherent in the PFB channel response, producing a flatter passband in the re-channelized output.